rule Â¶

AnyFeature `module-attribute` Â¶

```
AnyFeature = Union[ListFeature, MetricFeature, MapFeature]
```

logger `module-attribute` Â¶

```
logger = getLogger(__name__)
```

ControlFlowActionTypeEnum Â¶

Bases: `JavaEnumWrapper`

Rule Control flow action enum.

CONTINUE `class-attribute` `instance-attribute` Â¶

```
CONTINUE = auto()
```

NEXT_WORKFLOW_BLOCK `class-attribute` `instance-attribute` Â¶

```
NEXT_WORKFLOW_BLOCK = auto()
```

STOP_WORKFLOW `class-attribute` `instance-attribute` Â¶

```
STOP_WORKFLOW = auto()
```

Rule Â¶

```
Rule(pyjnius_java)
```

Bases: `JavaWrapper`

Represents a Rule in Pulse.

action_params `property` Â¶

```
action_params: Mapping[str, List[JavaWrapper]]
```

Get the map of parameters to return within the actions triggered by this rule.

Returns:

Type	Description
`Mapping[str, List[RuleActionParam]]`	The map of parameters to return within the actions triggered by this rule. The map indexes lists of parameters by the name of the action of the parameters.

actions `property` `writable` Â¶

```
actions: Set[str]
```

Get or set the list of actions to take when the rule evaluates to true.

app `property` Â¶

```
app: App
```

Get the rule's application

comment `property` `writable` Â¶

```
comment: str
```

Get or set the description of the rule.

condition `property` `writable` Â¶

```
condition: str
```

Get or set the RDL expression used as a condition for triggering the rule.

control_flow `property` `writable` Â¶

```
control_flow: ControlFlowActionTypeEnum
```

Get or set the rule control flow action to take after evaluating a rule.

desc `property` `writable` Â¶

```
desc: str
```

Get or set the user visible name of this rule.

explanation_template `property` `writable` Â¶

```
explanation_template: str
```

Get or set the velocity template used to generate the rule's explanation description.

fields `property` Â¶

```
fields: Set[str]
```

Get the schema fields used in this rule's condition and variables.

id `property` Â¶

```
id: str
```

Get the rule's id.

is_enabled `property` `writable` Â¶

```
is_enabled: bool
```

Get or set the rule's active status.

is_score_enabled `property` `writable` Â¶

```
is_score_enabled: bool
```

Get or set the flag that indicates if score and weight are enabled.

To set the flag to true, you must ensure that you first set the `Rule.scores` property, otherwise a validation error is thrown.

is_snapshot `property` Â¶

```
is_snapshot: bool
```

Get the rule's snapshotness.

managed_list_ids `property` Â¶

```
managed_list_ids: Set[str]
```

Get the Ids of the lists used by this rule.

order `property` Â¶

```
order: int
```

Get the order in which this rule should be evaluated.

original_id `property` Â¶

```
original_id: str
```

Get the rule's original id, if this is a snapshot rule.

parent_rules_project `property` Â¶

```
parent_rules_project: RulesProject
```

Get the rules project this rule belongs to.

parent_rules_project_snapshot_id `property` Â¶

```
parent_rules_project_snapshot_id: Optional[str]
```

Get the Id of the snapshot that this rule is associated to.

schema `property` Â¶

```
schema: Schema
```

Get the schema associated to this rule and which belongs to the parent RulesProject.

scores `property` `writable` Â¶

```
scores: Optional[List[float]]
```

Get or set an optional List that, if present, contains the scores that this rule should set when triggered.

The order in which the scores are present in the list should correspond to the order in which the possible target values are configured in the {@link AbstractRulesProjectModel#getSchema()}. The scores are expected to all be between Rule.model.MIN_RULE_SCORE and Rule.model.MAX_RULE_SCORE.

subclasses `class-attribute` `instance-attribute` Â¶

```
subclasses: List[JavaWrapper] = []
```

target_var_value `property` `writable` Â¶

```
target_var_value: Optional[str]
```

Get or set the value that should be assigned to the target variable.

variables `property` `writable` Â¶

```
variables: str
```

Get or set the RDL expression defining the variables used in the rule.

weight `property` `writable` Â¶

```
weight: int
```

Get or set the integer that defines the weight of this rule.

create `classmethod` Â¶

```
create(app: App, rules_project: RulesProject, name: str, condition: str, variables:
Optional[str] = None, comment: Optional[str] = None, actions: Optional[Set[str]] = None, is_enab
led: bool = True, control_flow: Optional[ControlFlowActionTypeEnum] = None, managed_lists_id: Op
tional[Set[str]] = None) -> Rule
```

Creates and returns a new Rule.

Parameters:

Name	Type	Description	Default
`app` Â¶	`App`	Application ds_api object.	*required*
`rules_project` Â¶	`RulesProject`	`RulesProject` where the rule will be created.	*required*
`name` Â¶	`str`	Name of the new rule.	*required*
`condition` Â¶	`str`	Condition implemented by the rule.	*required*
`variables` Â¶	`str`	Variables implemented to use in the rule's condition.	`None`
`comment` Â¶	`str`	Comment on the rule.	`None`
`actions` Â¶	`Set[str]`	Actions to take when the rule triggers. Default is an empty set.	`None`
`is_enabled` Â¶	`bool`	Boolean identifying whether the rule is enabled or not. Default is True.	`True`
`control_flow` Â¶	`ControlFlowActionTypeEnum`	Control flow action to take after evaluating a rule. Default is `ControlFlowActionTypeEnum.CONTINUE`	`None`
`managed_lists_id` Â¶	`Set[str]`	Set of list_ids used by the rule. Default is an empty set.	`None`

Returns:

Type	Description
Rule	New Rule.

duplicate Â¶

```
duplicate(new_desc: Optional[str] = None, new_control_flow: Optional[ControlFlowActionTypeEnum] = None, new_comment: Optional[str] = None, new_actions: Optional[Set[str]] = None, new_variables: Optional[str] = None, new_condition: Optional[str] = None, new_rules_project: Optional[RulesProject] = None) -> Rule
```

Duplicate this rule, optionally changing some of its attributes.

Parameters:

Name	Type	Description	Default
new_desc Â¶	str	The name of the duplicated rule. Defaults to the original name suffixed with '_duplicate'.	None
new_control_flow Â¶	Optional[ControlFlowActionTypeEnum]	The control flow to use in the duplicate. Defaults to the original.	None
new_actions Â¶	Optional[Set[str]]	The actions to used in the duplicate. Defaults to the original.	None
new_comment Â¶	str	The comment to use in the duplicate. Defaults to the original.	None
new_variables Â¶	str	The variables to use in the duplicate. Defaults to the original.	None
new_condition Â¶	str	The condition to use in the duplicate. Defaults to the original.	None
new_rules_project Â¶	RulesProject	The rules project to create the duplicate in. Defaults to the original.	None

Returns:

Type	Description
Rule	The duplicated rule.

Raises:

Type	Description
ValueError	Raised if a rule with the same name exists within the rules project.

from_dict classmethod Â¶

```
from_dict(app: App, rules_project: RulesProject, rule_as_dict: Dict[str, Any]) -> Rule
```

Create a Rule, given its dictionary representation.

Parameters:

Name	Type	Description	Default
app Â¶	App	The app in which to create the rule in.	*required*
rules_project Â¶	RulesProject	The rules project in which to create the rule in.	*required*
rule_as_dict Â¶	Dict[str, Any]	The dictionary representation of the rule. It must have the following keys: `name : str` `    Name of the new rule.` `condition : str` `    Condition implemented by the rule.` `variables : str` `    Variables implemented to use in the rule's condition.` `comment : str` `    Comment on the rule.` `actions : Set[str], optional` `    Actions to take when the rule triggers. Default is an empty` `    set.` `is_enabled : bool, optional` `    Boolean identifying whether the rule is enabled or not.` `    Default is True.` `control_flow : str,` `    The name of the control flow action to take after evaluating` `    a rule. It must be the name of a ControlFlowActionTypeEnum` `    member. Default is `CONTINUE` `target_var_value : str, optional` ` The value that should be assigned to the target variable.` `is_score_enabled : bool, optional` ` The flag that indicates if score and weight are enabled.` `scores : Sequence[float], optional` ` The scores that this rule should set when triggered.` `weight : str, optional` ` The integer that defines the weight of this rule` `explanation_template : str, optional` ` The velocity template used to generate the rule's` ` explanation description.`	*required*

Returns:

Type	Description
Rule	The Rule created from the given dictionary representation.

to_dict Â¶

```
to_dict() -> Dict[str, Any]
```

Generate a description of this rule in dictionary format.

Returns:

Type	Description
`Dict[str, Any]`	A description of this rule's main attributes.

update Â¶

```
update(**kwargs: Mapping[str, Any]) -> None
```

Update several properties of this Rule at once.

Danger

This method doesn't check whether the properties you pass as a kwarg is one of the existing modifiable ones, so it's up to the caller to make sure the keys are correct.

Parameters:

Name	Type	Description	Default
**`**kwargs` Â¶	`Mapping[str, Any]`	Map between rule property names and the values we want to update them to	`{}`

Raises:

Type	Description
`TypeError`	If any of the given property values aren't of the required type